

III. Lire et écrire des signaux analogiques

Faire varier la luminosité d'une LED en fonction de la position d'un potentiomètre.

- Câblage :
 - Placer le potentiomètre et la LED sur le breadboard.
 - Potentiomètre :
 - Broche 1 : Connectée à la masse
 - Broche 2 : Connectée à une broche analogique (ex : A0)
 - Broche 3 : Connectée à la tension d'alimentation (5 V).
 - LED :
 - Connecter la broche longue (anode, +) à une broche PWM (ex. D9) via une résistance de 220 Ω .
 - Connecter la broche courte (cathode, -) à la masse

- Le programme :

```
void setup() {
    pinMode(9, OUTPUT);      // Broche PWM 9 en mode sortie
}

void loop() {
    int potValue = analogRead(A0);
    // Lire la valeur du potentiomètre, entre 0 et 1023.
    int brightness = map(potValue, 0, 1023, 0, 255);
    // Convertir analogRead(A0) en valeur PWM (0 à 255)
    analogWrite(9, brightness); // génère un signal PWM pour
    // contrôler la luminosité de la LED
    delay(10);                 // délai pour stabiliser la lecture
}
```

- Exercices à faire :
 1. Lire deux potentiomètres pour contrôler la couleur d'une LED RGB.
 2. Contrôler la vitesse d'un moteur DC avec un potentiomètre

IV. Communication avec les périphériques

A. UART

- Exemple 1: Envoi et réception de données entre l'Arduino et le moniteur série
 - Exécuter le code suivant :

```
void setup() {
    Serial.begin(9600);

    // Initialise a communication série à 9600 bauds

    Serial.println("Entrez un message :");
    // Envoyer un message au moniteur série
}

void loop() {
    if (Serial.available() > 0) {
        // Vérifier si des données sont reçues
        String message = Serial.readString();
    }
}
```

```

        // Lire le message reçu
        Serial.print("Vous avez écrit : ");
        Serial.println(message);
    }
}

```

- Exemple 2 : Contrôler une LED depuis le moniteur série

- Câblage :

- Placer la résistance et la LED sur le breadboard.
 - Cathode de la LED (borne courte) → Résistance de 220 Ω .
 - Autre extrémité de la résistance → GND de l'Arduino.
 - Broche numérique 13 de l'Arduino → Anode de la LED.

- Programme :

```

void setup() {
    Serial.begin(9600);
    pinMode(13, OUTPUT);
    digitalWrite(13, LOW); // Éteint la LED au démarrage
    Serial.println("Tapez 'ON', 'OFF' ou 'CLIGNOTER'.");
}

void loop() {
    if (Serial.available()) {
        String command = Serial.readString(); // Lit la commande
        command.trim(); // Supprime les espaces inutiles
        if (command == "ON") {
            digitalWrite(13, HIGH);
            Serial.println("LED allumée");
        }
        else if (command == "OFF") {
            digitalWrite(13, LOW);
            Serial.println("LED éteinte");
        }
        else if (command == "CLIGNOTER") {
            Serial.println("LED clignotante");
            for (int i = 0; i < 5; i++) { // clignoter la LED 5 fois
                digitalWrite(13, HIGH);
                delay(500);
                digitalWrite(13, LOW);
                delay(500);
            }
        }
        else {
            Serial.println("Commande non reconnue.");
        }
    }
}

```

- Exercices :
 1. Envoyer des données d'un capteur (ex : potentiomètre) vers le moniteur série.
 2. Envoyer "Hello World" via le port série

B. Scanner les périphériques connectés sur le bus I2C

- Câblage :
 - Ecran I2C : SSD1306 OLED
 -  VCC → 5V
 -  GND → GND
 -  SCL → A5
 -  SDA → A4

- Programme :


```
#include <Wire.h>

void setup() {
    Serial.begin(9600);
    //Initialise la communication avec le moniteur série
    Wire.begin();
    Serial.println("Scan I2C...");
}

void loop() {
    byte error, address;
    int nDevices = 0;

    for(address = 1; address < 127; address++ ) {
        Wire.beginTransmission(address);
        error = Wire.endTransmission();

        if (error == 0) {
            Serial.print("Périphérique trouvé à 0x");
            Serial.println(address, HEX);
            nDevices++;
        }
    }

    if (nDevices == 0)
        Serial.println("Aucun périphérique trouvé");
    delay(5000);
}
```

- Exercices :
 1. Trouver l'adresse I2C de l'écran OLED puis afficher "Bonjour Arduino".
 2. Lire les données d'un capteur I2C.
 3. Gérer plusieurs périphériques I2C sur le même bus.
 4. Reprendre ces exercices en utilisant des bibliothèques (I2CScanner, par exemple).

C. SPI pour contrôler une matrice LED 8x8

- Câblage
 - Le module MAX7219 (contrôle typiquement des matrices 8x8. Si la matrice est de 32x8, cela signifie généralement que vous avez 4 modules MAX7219 connectés en cascade) :
 -  VCC → 5V
 -  GND → GND
 -  DIN → MOSI (Pin 11)
 -  CS → SS (Pin 10)
 -  CLK → SCK (Pin 13)
- Programme :

```
#include <SPI.h>

const int CS = 10; // sélection de la broche du module MAX7219

void sendData(byte reg, byte data) {
    digitalWrite(CS, LOW);
    SPI.transfer(reg);
    SPI.transfer(data);
    digitalWrite(CS, HIGH);
}

void setup() {
    SPI.begin();
    pinMode(CS, OUTPUT);

    // Initialisation du MAX7219
    sendData(0x0C, 0x01); // Activer l'affichage
    sendData(0x09, 0x00); // Mode normal (pas de décodage)
    sendData(0x0B, 0x07); // Activer les 8 lignes
    sendData(0x0A, 0x08); // Régler l'intensité (0x00 à 0x0F)
    sendData(0x0F, 0x00); // Désactiver le mode test
}

void loop() {
    sendData(3, 0xFF); // Allumer la 3e ligne
    delay(500);
    sendData(3, 0x00); // Éteindre la 3e ligne
    delay(500)
}
```

- Exercices :
 1. Afficher un texte défilant sur la matrice.
 2. Afficher la position d'un potentiomètre sur la matrice
 3. Animer des motifs (clignotements, rotation,...)
 4. Contrôler plusieurs matrices LED en cascade.

V. Passage au hardware

- Installation de l'IDE Arduino, configuration et prise en main.
- Reprendre les exemples simulés en utilisant le hardware.